

documelt

[![npm
version](https://img.shields.io/npm/v/documelt.svg)](https://www.npmjs.com/package/documelt)
[![License: MIT](https://img.shields.io/badge/License-MIT-blue.svg)](https://opensource.org/licenses/MIT)
[![GitHub](https://img.shields.io/github/stars/NAOHIDEOTA/documelt?style=social)](<https://github.com/NAOHIDEOTA/documelt>)

****Rust + WebAssembly**** によるブラウザ向けドキュメントテキスト抽出ライブラリ。

サーバーに依存せず、****ブラウザ上で**** PDF・Word・Excel・PowerPoint からテキストを高速に抽出します。

Features

- **ブラウザ完結** — サーバーサイド不要。WASM がブラウザ内でテキスト抽出を実行
- **高速** — Rust コンパイルによるネイティブ級パフォーマンス (PDF は 0.2 秒、DOCX/XLSX は数 ms)
- **ページ単位の抽出** — PDF→ページ、PPTX→スライド、XLSX→シートごとに配列で返却
- **Web Worker 対応** — メインスレッドをブロックしない非同期処理
- **メタデータ付き** — ファイル名、サイズ、ページ数、文字数、処理時間を自動計測
- **TypeScript** — 完全な型定義付き

Supported Formats

Format	Engine	Output
PDF	[pdf-extract](https://crates.io/crates/pdf-extract)	ページごとのテキスト配列
DOCX	ZIP + XML parse	ドキュメント全体のテキスト
XLSX	[calamine](https://crates.io/crates/calamine)	シートごとのテキスト配列 (タブ区切り)
PPTX	ZIP + XML parse	スライドごとのテキスト配列
TXT	UTF-8 decode	そのままのテキスト

> **Note:** スキャン PDF やアウトライン化された PDF (テキストがベクターパスとして埋め込まれたもの) はテキスト抽出できません。OCR が必要なケースはサーバーサイドでの処理を推奨します。

Installation

```
```bash
npm install documelt
```
```

Quick Start

```
```typescript
import { extractFromFile } from 'documelt';

// ファイル選択イベントから
const input = document.querySelector<HTMLInputElement>('#file');
input.addEventListener('change', async () => {
 const result = await extractFromFile(input.files[0]);

 if (result.success) {
 // ページ単位の配列
 console.log(result.texts); // ['Page 1 text...', 'Page 2 text...', ...]

 // 全文を1つの文字列として
 console.log(result.texts.join('¥n'));

 // メタデータ
 console.log(result.meta);
 // { filename: 'doc.pdf', extension: 'pdf', size: 204800, pages: 5, characters: 12000, time:
0.15 }
 } else {
 console.error(result.error);
 }
});
```
```

WASM の初期化は初回呼び出し時に自動で行われます。`init()` を明示的に呼ぶ必要はありません。

Usage

Browser (Main Thread)

```
```typescript
import { extract, extractFromFile, isSupported } from 'documelt';

// File API から
const result = await extractFromFile(file);

// Uint8Array + 拡張子を直接指定
const result = await extract(uint8Array, 'pdf', 'report.pdf');

// 対応フォーマット判定
isSupported('report.pdf'); // true
isSupported('image.png'); // false
```
```

Browser (Web Worker)

大きなファイルの処理でメインスレッドのブロックを避けたい場合に使用します。

```
```typescript
import { DocumeltWorker } from 'documelt';

const worker = new Worker(
 new URL('documelt/worker', import.meta.url),
 { type: 'module' }
);
const client = new DocumeltWorker(worker);

const result = await client.extractFromFile(file);
console.log(result.texts);

// 不要になったら終了
client.terminate();
```

``

### ### Node.js

```typescript

```
import { readFileSync } from 'fs';
```

```
import { initWithBytes, extract } from 'documelt';
```

// Node.js では WASM バイナリを手動で初期化

```
initWithBytes(readFileSync('node_modules/documelt/pkg/documelt_bg.wasm'));
```

```
const data = new Uint8Array(readFileSync('./document.pdf'));
```

```
const result = await extract(data, 'pdf', 'document.pdf');
```

```
console.log(result.texts.join('\n'));
```

```

## ## API Reference

### ### `extractFromFile(file: File): Promise<ExtractionResult>`

ブラウザの `File` オブジェクトからテキストを抽出します。

### ### `extract(data: Uint8Array, extension: SupportedFormat, filename?: string): Promise<ExtractionResult>`

バイナリデータとファイル拡張子を指定してテキストを抽出します。

### ### `isSupported(fileName: string): boolean`

ファイル名から対応フォーマットかどうかを判定します。

### ### `init(wasmUrl?: string | URL): Promise<void>`

WASM モジュールを手動で初期化します。通常は自動初期化されるため呼ぶ必要はありません。初期化タイミングを制御したい場合（プリロード等）に使用します。

### ### `initWithBytes(wasmBytes: BufferSource): void`

WASM バイナリを直接渡して同期的に初期化します。Node.js 環境向け。

### ### `DocumeltWorker`

Web Worker 経由で抽出を行うクライアントクラス。

```
```typescript
const client = new DocumeltWorker(worker);
await client.extract(data, 'pdf', 'report.pdf');
await client.extractFromFile(file);
client.terminate();
```
```

### ### `SUPPORTED\_FORMATS`

対応フォーマットの配列: `['pdf', 'docx', 'xlsx', 'pptx', 'txt']`

## ## Types

```
```typescript
interface ExtractionResult {
  texts: string[];          // ページ/シート/スライドごとのテキスト配列
  success: boolean;         // 抽出成功フラグ
  error: string | null;     // エラーメッセージ (成功時 null)
  meta: ExtractionMeta;     // メタデータ
}

interface ExtractionMeta {
  filename: string;         // ファイル名
  extension: SupportedFormat; // 拡張子
  size: number;             // ファイルサイズ (bytes)
  pages: number;            // ページ数 (PDF/PPTX/XLSX)
  characters: number;       // 総文字数
  time: number;             // 処理時間 (秒)
}

type SupportedFormat = 'pdf' | 'docx' | 'xlsx' | 'pptx' | 'txt';
```
```

## ## Performance

WASM バイナリサイズ: **\*\*~1.2MB\*\***

| Format | File Size | Pages     | Characters | Time   |
|--------|-----------|-----------|------------|--------|
| -----  | -----     | -----     | -----      | -----  |
| TXT    | 209B      | 1         | 75         | 0.001s |
| PDF    | 2.3MB     | 31        | 4,802      | 0.21s  |
| DOCX   | 30KB      | 1         | 4,461      | 0.002s |
| XLSX   | 18KB      | 3 sheets  | 4,492      | 0.004s |
| PPTX   | 44MB      | 31 slides | 4,059      | 0.02s  |

## ## Limitations

- **\*\*スキャン PDF / アウトライン PDF\*\*** — テキストがベクターパスや画像として埋め込まれた PDF は抽出不可（OCR が必要）
- **\*\*パスワード保護ファイル\*\*** — 暗号化されたファイルは非対応
- **\*\*ファイルサイズ\*\*** — ブラウザメモリに制約あり。10MB 以下を推奨

## ## Contributing

[GitHub Issues](<https://github.com/NAOHIDEOTA/documelt/issues>) でバグ報告・機能リクエストを受け付けています。

## ## License

[MIT]([LICENSE](#)) - Copyright (c) 2026 NAOHIDEOTA